Komponenttester med bUnit

Et lite eksempel som tester at en komponent sender riktig verdi gjennom ValueChanged

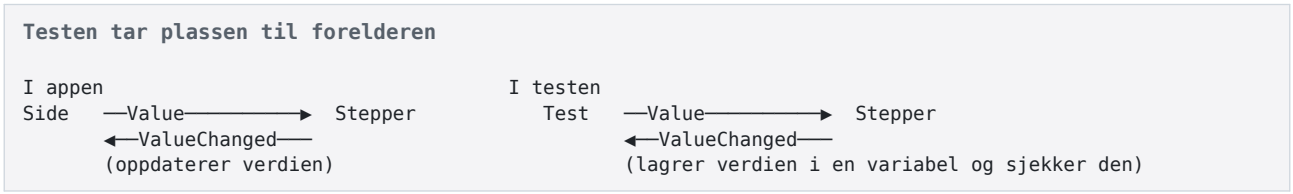
Dokumentet viser steg for steg hvordan en Blazor-komponent testes med bUnit. Eksempelet bruker en liten, oppdiktet komponent, `Stepper`, med en pluss- og en minusknapp. Den har samme mønster som en komponent med `Value` og `ValueChanged`: den endrer aldri sin egen verdi, men sier fra til forelderen. Når mønsteret sitter, kan det brukes på hvilken som helst komponent med `@bind-Value`.

Hva bUnit gjør

En vanlig enhetstest kaller en metode og sjekker svaret. En komponent er vanskeligere å teste, fordi den lager HTML og reagerer på klikk. bUnit løser det ved å kjøre komponenten i minnet, uten nettleser og uten server. Testen kan da:

- **rendre** komponenten med de parameterne testen velger,
- **finne** elementer i HTML-en som komponenten lager, med CSS-selektorer som `button.plus`,
- **utløse hendelser**, for eksempel klikk og endring i et inndatafelt,
- **sjekke resultatet**: hva som ble sendt gjennom en `EventCallback`, og hva HTML-en viser etterpå.

Testen erstatter forelderen. I appen er det en side som gir komponenten `Value` og tar imot `ValueChanged`. I testen er det testen selv som gjør begge deler.



Komponenten som testes

`Stepper` viser et tall med en knapp på hver side. Pluss sender `Value + 1`, og minus sender `Value - 1`. Under `Min` sendes ingenting. Knappene har klassene `plus` og `minus`, slik at testen kan finne dem.

Components/Shared/Stepper.razor

```
@* A small component that counts up and down. It never changes its own Value,
   but tells the parent about the new value through ValueChanged *@

<div class="stepper">
    <button class="minus" @onclick="Decrease">–</button>
    <span class="value">@Value</span>
    <button class="plus" @onclick="Increase">+</button>
</div>

@code {
    [Parameter] public int Value { get; set; }
    [Parameter] public EventCallback<int> ValueChanged { get; set; }

    // The lowest allowed value
    [Parameter] public int Min { get; set; } = 0;

    private Task Increase() => ValueChanged.InvokeAsync(Value + 1);

    private Task Decrease()
    {
        // Below the minimum: nothing is sent to the parent
        if (Value <= Min)
            return Task.CompletedTask;

        return ValueChanged.InvokeAsync(Value - 1);
    }
}
```

I en side brukes den slik. `@bind-Value` kobler `Value` (ned) og `ValueChanged` (opp) i én kortform:

Bruk i en side

```
<Stepper @bind-Value="players" Min="1" />

@code {
    private int players = 2;
}
```

Oppsett

Testprosjektet trenger pakken `bunit`, xUnit og en referanse til prosjektet med komponenten. I `AK2Test.csproj` er alt dette allerede på plass:

AK2Test.csproj (finnes allerede)

```
<PackageReference Include="bunit" Version="2.11.3" />
<PackageReference Include="xunit" Version="2.9.3" />
...
<ProjectReference Include="..\AK2\AK2.csproj" />
```

Testklassen arver fra `BunitContext`. Den gir testen en liten «Blazor i minnet» med metoden `Render` og en egen tjenestebeholder. xUnit lager en ny instans av klassen for hver test, så testene påvirker ikke hverandre.

> **bUnit 1 og 2 har ulike navn.** Prosjektet bruker bUnit 2. Der heter basisklassen `BunitContext`, og metoden heter `Render`. Eldre eksempler på nettet bruker bUnit 1, der de heter `TestContext` og `RenderComponent`. Samme idé, andre navn.

Den første testen, steg for steg

Testen sjekker at et klikk på pluss sender `4` når `Value` er `3`. Den følger mønsteret Arrange, Act, Assert.

AK2Test/StepperTests.cs

```
[Fact]
public void ClickPlus_SendsValuePlusOne()
{
    // Arrange
    int? receivedValue = null;

    var cut = Render<Stepper>(parameters => parameters
        .Add(p => p.Value, 3)
        .Add(p => p.ValueChanged, (int newValue) => receivedValue = newValue));

    // Act
    cut.Find("button.plus").Click();

    // Assert
    Assert.Equal(4, receivedValue);
}
```

Arrange: forbered

`int? receivedValue = null;` er en variabel som skal ta imot verdien komponenten sender. Den starter som `null`, slik at testen kan se forskjell på «ingenting sendt» og «0 sendt».

`Render<Stepper>(...)` rendrer komponenten. Inne i lambdaen gis parameterne, akkurat som attributter i en side:

- `.Add(p => p.Value, 3)` tilsvarer `Value="3"`.
- `.Add(p => p.ValueChanged, (int newValue) => receivedValue = newValue)` tilsvarer `ValueChanged="..."`. bUnit gjør lambdaen om til en `EventCallback<int>`. Når komponenten kaller `ValueChanged.InvokeAsync(4)`, kjører lambdaen, og `receivedValue` blir `4`.

`p => p.Value` peker på selve egenskapen i komponenten. Skrives navnet feil, får du en kompileringsfeil i stedet for en test som feiler i det stille.

`cut` er resultatet av `Render`. Navnet er en vanlig forkortelse for «component under test».

Act: gjør handlingen

`cut.Find("button.plus")` finner knappen med en CSS-selektor, slik som i en stilark. `.Click()` utløser `@onclick` på samme måte som et ekte klikk. Finnes ikke elementet, kaster `Find` et unntak, og testen feiler med en tydelig melding.

Assert: sjekk resultatet

`Assert.Equal(4, receivedValue)` sjekker at komponenten sendte riktig verdi. Det første argumentet er forventet verdi, og det andre er det som faktisk skjedde.

Hele testfilen

Filen har fire tester: to for normal bruk, ett kanttilfelle og én som sjekker at komponenten ikke endrer sin egen verdi.

AK2Test/StepperTests.cs (del 1 av 2)

```
using AK2.Components.Shared;
using Bunit;

namespace AK2Test
{
    // BunitContext gives each test its own small "Blazor in memory"
    public class StepperTests : BunitContext
    {
        [Fact]
        public void ClickPlus_SendsValuePlusOne()
        {
            // Arrange
            int? receivedValue = null;

            var cut = Render<Stepper>(parameters => parameters
                .Add(p => p.Value, 3)
                .Add(p => p.ValueChanged, (int newValue) => receivedValue = newValue));

            // Act
            cut.Find("button.plus").Click();

            // Assert
            Assert.Equal(4, receivedValue);
        }

        [Fact]
        public void ClickMinus_SendsValueMinusOne()
        {
            // Arrange
            int? receivedValue = null;

            var cut = Render<Stepper>(parameters => parameters
                .Add(p => p.Value, 3)
                .Add(p => p.ValueChanged, (int newValue) => receivedValue = newValue));

            // Act
            cut.Find("button.minus").Click();

            // Assert
            Assert.Equal(2, receivedValue);
        }
```

AK2Test/StepperTests.cs (del 2 av 2, fortsetter rett under)

```
        [Fact]
        public void ClickMinus_AtMinimum_SendsNothing()
        {
            // Arrange
            int? receivedValue = null;

            var cut = Render<Stepper>(parameters => parameters
                .Add(p => p.Value, 0)
                .Add(p => p.Min, 0)
                .Add(p => p.ValueChanged, (int newValue) => receivedValue = newValue));

            // Act
            cut.Find("button.minus").Click();

            // Assert
            Assert.Null(receivedValue);
        }

        [Fact]
        public void ClickPlus_DoesNotChangeOwnValue()
        {
            // Arrange
            var cut = Render<Stepper>(parameters => parameters
                .Add(p => p.Value, 3)
                .Add(p => p.ValueChanged, (int newValue) => { }));

            // Act
            cut.Find("button.plus").Click();

            // Assert: the component only reports the new value. The parent owns Value
            Assert.Equal(3, cut.Instance.Value);
            Assert.Equal("3", cut.Find("span.value").TextContent);
        }
    }
}
```

Test	Hva den viser
`ClickPlus_SendsValuePlusOne`	Normal bruk: pluss sender `Value + 1`
`ClickMinus_SendsValueMinusOne`	Normal bruk: minus sender `Value - 1`
`ClickMinus_AtMinimum_SendsNothing`	Kanttilfelle: ved minimum sendes ingenting. `Assert.Null` viser at lambdaen aldri kjørte
`ClickPlus_DoesNotChangeOwnValue`	Komponenten endrer ikke sin egen parameter. `cut.Instance` gir tilgang til selve komponentobjektet, og teksten på skjermen er fortsatt 3

> **Hvorfor den siste testen er nyttig:** En komponent som skriver `Value++` selv, ville bestått den første testen hvis den også kalte `ValueChanged`. Men den bryter regelen om at forelderen eier verdien. Den siste testen fanger akkurat den feilen.

Byggeklossene

I testen	Betyr
`BunitContext`	Basisklassen som gir `Render` og `Services`
`Render<T>(p => p.Add(...))`	Rendrer komponenten `T` med parametere
`cut.Find("selektor")`	Finner ett element med en CSS-selektor. Kaster unntak hvis det ikke finnes
`cut.FindAll("selektor")`	Finner alle elementer som passer, for eksempel alle knapper
`.Click()`	Utløser `@onclick`
`.Change("5")`	Endrer verdien i et inndatafelt og utløser `@onchange`, eller `@bind`, som bruker `onchange`
`.Input("5")`	Som `Change`, men for `@oninput` og `@bind:event="oninput"`
`.TextContent`	Teksten inne i et element
`cut.Markup`	Hele HTML-en komponenten har laget, nyttig å skrive ut når en test feiler
`cut.Instance`	Selve komponentobjektet, for å lese parametere og felt
`Services.AddSingleton(...)`	Registrerer tjenester komponenten henter med `@inject`

Når komponenten har et inndatafelt

Mange komponenter lar brukeren skrive inn en verdi og trykke på en knapp. Da er Act-delen to steg: først fylles feltet ut, så klikkes knappen. Et oppdiktet eksempel med et felt og en «Lagre»-knapp:

```
Act med et inndatafelt

// Act
cut.Find("input").Change("5");
cut.Find("button").Click();
```

`Change` passer når feltet bruker `@bind`, som oppdaterer verdien når feltet mister fokus. Bruker feltet `@bind:event="oninput"`, brukes `Input` i stedet. Har komponenten flere knapper, trengs en mer presis selektor, for eksempel en klasse eller `button[type=button]`.

Når komponenten bruker @inject

bUnit lager komponenten slik Blazor gjør. Henter komponenten en tjeneste med `@inject`, må tjenesten finnes i testen også, ellers feiler `Render` med en melding om at tjenesten ikke er registrert. Tjenesten registreres før `Render`:

```
Registrere en tjeneste i testen

// Arrange
Services.AddSingleton(new SomeService());

var cut = Render<MyComponent>(parameters => parameters ...);
```

Trenger tjenesten en `HttpClient`, kan den få en med en tenkt adresse, for eksempel `new HttpClient { BaseAddress = new Uri("http://localhost/") }`. Kallene vil da feile. Om det er greit, avhenger av om komponenten fanger feilen. En komponent som bare viser data og sier fra med hendelser, trenger ingen tjenester, og er derfor enklest å teste.

Kjøre testene

1. Legg `StepperTests.cs` i testprosjektet, og `Stepper.razor` i `Components/Shared` i appen, hvis du vil prøve eksempelet som det er.
2. Bygg løsningen (**Build → Rebuild Solution**).
3. Åpne **Test → Test Explorer** og trykk **Run All Tests**.
4. Grønne haker betyr at testen passerte. En rød test viser forventet og faktisk verdi. `cut.Markup` i en `Console.WriteLine` eller i feilsøkeren viser hva komponenten faktisk laget.

> **Prøv å få testen til å feile.** Bytt `Value + 1` med `Value + 2` i `Stepper`, og kjør testene igjen. `ClickPlus_SendsValuePlusOne` skal da feile med «Expected: 4, Actual: 5». Slik ser man at testen faktisk sjekker noe.

Feilene i det første forsøket på DiceRatingTests

Første forsøk på å bruke eksempelet på `DiceRating` ga både kompileringsfeil og en test som ikke testet det den skulle. Tabellen viser hver feil, hvorfor den oppsto, og løsningen.

Feil	Hvorfor	Løsning
Testklassen het `DiceRating`, som komponenten	`Render<DiceRating>` pekte på testklassen. Kompilatoren valgte en annen `Render`, og feilene handlet om `RenderTreeBuilder` og `Type`	Gi klassen et eget navn, `DiceRatingTests`, og legg til `using AK2.Components.Shared;`
Rename i Visual Studio endret også `Render<DiceRating>`	Visual Studio trodde typen var den samme klassen, og laget `Render<DiceRatingTests>`	Sett typen tilbake til `Render<DiceRating>`
`Find("button.plus")` fra eksempelet	DiceRating har ingen knapp med klassen `plus`	Bruk selektorer som finnes i markupen: `Find("input")` og `Find("button")`
`ReadOnly` ble ikke satt	Standardverdien er `true`, så feltet og knappen ble ikke vist	`.Add(p => p.ReadOnly, false)` i parameterlista
Startverdi 3 og forventet verdi 3	En komponent som sender den gamle `Value` i stedet for den nye, ville også bestått	Skriv en ny verdi i feltet, og forvent den: `Change("5")` og `Assert.Equal(5, ...)`
`await ValueChanged.InvokeAsync(newValue)` i testen	`ValueChanged` finnes i komponenten, ikke i testen. Det er komponenten som skal kalle den, og det er det testen sjekker	Gjør det brukeren gjør: `cut.Find("input").Change("5")`
Lokal variabel `int newValue`	Samme navn som parameteren i lambdaen. C# tillater ikke det	Fjern variabelen
`Services.AddSingleton<GameApiClient>()`	Konstruktøren trenger en `HttpClient`, som ikke er registrert i testen	Registrer en ferdig instans: `new GameApiClient(new HttpClient { BaseAddress = ... })`
`async Task` uten `await`	Gir advarselen CS1998	`public void`
`Change(value)` med en `int`	Nettleseren sender alltid tekst, og `@bind` gjør teksten om til tall	`Change(value.ToString())`
Skrivefeil i kommentarene	«startvalue», «i sent», «recieved»	«start value», «is sent», «received»

De ferdige testene

Den første testen sjekker normal bruk: at den nye verdien sendes. Den andre sjekker kanttilfellene: at ingenting sendes når verdien er utenfor 1–6.

AK2Test/DiceRatingTests.cs (del 1 av 2)

```
using AK2.Client;
using AK2.Components.Shared;
using Bunit;
using Microsoft.Extensions.DependencyInjection;

namespace AK2Test
{
    public class DiceRatingTests : BunitContext
    {
        [Fact]
        public void SaveAsync_SendsValue()
        {
            // Arrange
            int? receivedValue = null;

            // Adding GameApiClient since total rating is fetched in OnInitializedAsync
            Services.AddSingleton(new GameApiClient(
                new HttpClient { BaseAddress = new Uri("http://localhost/") }));

            var cut = Render<DiceRating>(parameters => parameters
                .Add(p => p.Value, 3)
                .Add(p => p.ReadOnly, false)
                .Add(p => p.ValueChanged, (int newValue) => receivedValue = newValue));

            // Act
            cut.Find("input").Change("5");
            cut.Find("button").Click();

            // Assert
            Assert.Equal(5, receivedValue);
        }
```

AK2Test/DiceRatingTests.cs (del 2 av 2, fortsetter rett under)

```
        [Theory]
        [InlineData(7)]
        [InlineData(0)]
        [InlineData(-1)]
        public void SaveAsync_DoesNotSendValue_IfValueIsInvalid(int value)
        {
            // Arrange
            int? receivedValue = null;

            // Adding GameApiClient since total rating is fetched in OnInitializedAsync
            Services.AddSingleton(new GameApiClient(
                new HttpClient { BaseAddress = new Uri("http://localhost/") }));

            var cut = Render<DiceRating>(parameters => parameters
                .Add(p => p.Value, 3)
                .Add(p => p.ReadOnly, false)
                .Add(p => p.ValueChanged, (int newValue) => receivedValue = newValue));

            // Act
            cut.Find("input").Change(value.ToString());
            cut.Find("button").Click();

            // Assert
            Assert.Null(receivedValue);
        }
    }
}
```